Settling 648 unsolved BOBILib instances: published optima, new optimality proofs and infeasibility certificates

Brian J. Connolly

Independent Researcher

ABSTRACT

The Bilevel Optimization (Benchmark) Instance Library (BOBILib) holds 2,654 mixed-integer bilevel linear problems, and its metadata of 18 October 2026 lists 1,594 of them as feasible or open (here: unsolved). Its statuses come from two general-purpose solvers run for one hour each. This paper asks which of these instances can be settled, that is, proven optimal or proven to have no bilevel feasible point. Most of them share structure that general solvers do not use: 1,254 are packing interdiction games, 60 are clique interdiction problems dense enough for extremal graph theory to apply, and most of the rest are built from models of the Mixed Integer Programming Library (MIPLIB), whose published single-level optima can serve as lower bounds. Published work has already solved 528 of the 1,594. Methods built for these structures give 80 new optimality proofs, among them all 16 knapsack interdiction instances that were open in the literature and 57 of the 60 clique interdiction instances. They show that 40 instances have no bilevel feasible point, because the follower's problem is infeasible or unbounded for every leader choice (38 of these certificates are checked in exact arithmetic); they give a verified point for 230 instances where BOBILib lists none, 168 of them the first point known anywhere; and they improve on every published value for 469 instances. In all, 648 instances (41%) are now settled, 528 by published work and 120 here. The solution files use BOBILib's format and pass a separate verifier modeled on BOBILib's checker, the certificates behind the bounds are published, and the main pruning lemmas of the new exact search for knapsack interdiction are proved in the Lean theorem prover. An updated status file, the solution files and the certificates are released as a data package.

Keywords: bilevel optimization, interdiction games, knapsack interdiction, clique interdiction, benchmark library, optimality certificates, formal verification

1. INTRODUCTION

In a bilevel optimization problem a leader decides first and a follower then solves its own optimization problem, knowing the leader's decision. The leader's cost depends on the follower's answer, so the leader has to anticipate it (Figure 1). This is the leader-follower game of Stackelberg,[1] and it is the natural model of attack and defense: an interdictor removes or degrades parts of a system within a budget, and an operator then makes the best use of what is left.[2,3] Mixed-integer bilevel linear problems (MIBLPs), in which both levels are mixed-integer linear programs, are hard in theory and in practice. They are Σ_2^p-hard in general,[4] and even the bilevel knapsack problem with interdiction constraints is Σ_2^p-complete.[5] A textbook and a recent survey cover the field.[6,7]

Benchmark libraries focus research effort. For single-level mixed-integer programming, the Mixed Integer Programming Library (MIPLIB)[8–10] records for every instance whether it is solved, and its open instances have long been targets for new methods. BOBILib, the Bilevel Optimization (Benchmark) Instance Library of Thürauf, Kleinert, Ljubić, Ralphs and Schmidt,[11,12] plays the same role for MIBLPs. It collects 2,654 instances from the literature in one file format, with solution files, a feasibility checker and a status per instance. Its metadata of 18 October 2026 lists 1,027 instances as optimal, 33 as infeasible, 1,175 as feasible (a point is known, but not proven optimal) and 419 as open (no point is known). The statuses come from runs of two general-purpose bilevel solvers, MibS[13,14] and the solver of Fischetti et al. in its setting MIX++, or HC++ where MIX++ does not apply,[15,16] each for one hour on one thread. They were fixed when the instances were collected, and some of the specialized results discussed below appeared later.

This paper asks a simple question: which of the 1,594 feasible or open instances (below, *unsolved*) can be *settled*, that is, proven optimal or proven to have no bilevel feasible point, and are there methods that settle many of them at once? The goals of the work were to:

Further author information: E-mail: EMAIL

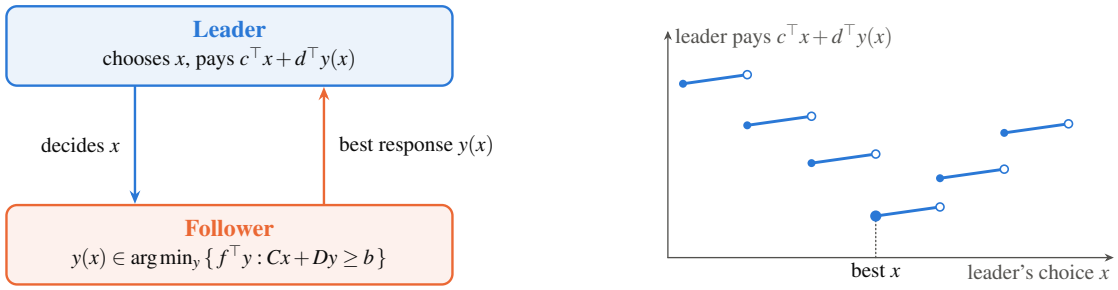


Figure 1. The bilevel game. The leader chooses x and pays $c^\top x + d^\top y(x)$, where $y(x)$ is the follower's best response to x. The leader's cost jumps wherever the follower's answer changes (right: schematic).

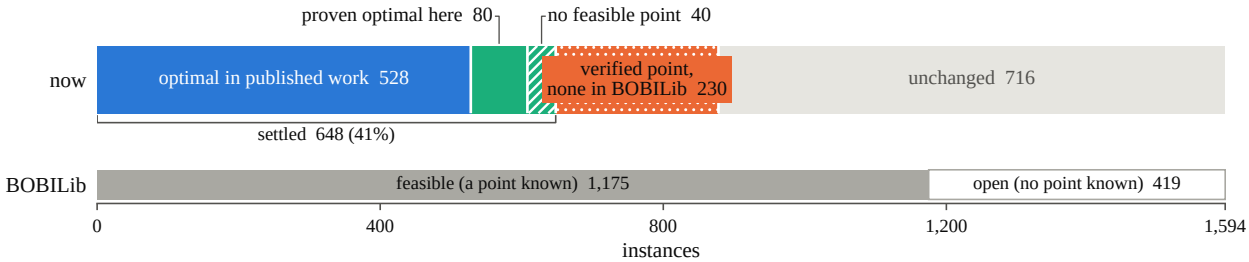


Figure 2. Where BOBILib's 1,594 feasible or open instances stand. 648 (41%) are settled: 528 were already optimal in published work, 80 are proven optimal here and 40 have no bilevel feasible point. Another 230 get a verified point where BOBILib lists none (for 168 of them the first point anywhere) and 716 are unchanged. The thin bar shows BOBILib's own split (1,175 feasible, 419 open).

- reconcile BOBILib's statuses with the per-instance results published since the instances were collected;
- find structure shared by many of the remaining instances, and exploit it with methods that settle whole families rather than single instances;
- make every claim checkable by a reader who does not trust my code: verified solution files in BOBILib's own format, a certificate for every bound, and machine-checked proofs where the argument is new.

The main findings are these (Figure 2).

- Published work had already solved 528 of the 1,594 instances: 428 with methods built for knapsack interdiction[17,18] and 97 with the decision diagrams of Lozano, Bergman and Cire.[19] Reconciling the statuses with these results settles more instances than any method of this paper.
- A new exact search proves all 16 knapsack interdiction instances that were open in the literature optimal, the slowest in 10.0 seconds. In all of them the optimum equals the trivial upper bound, so the whole difficulty lies in the lower bound. The main pruning lemmas of the search are proved in Lean.
- Two theorems of extremal graph theory, Mantel's and Brouwer's, together with partition certificates and 170 small cases decided by exact solvers, prove 57 of the 60 edge clique interdiction instances optimal, at values 3 and 4 where BOBILib lists 5 to 13.
- In 40 instances the follower's problem is infeasible or unbounded for every leader choice (38 certificates checked in exact arithmetic), so they have no bilevel feasible point at all. Another 230 instances get a verified point where BOBILib lists none, 168 of them the first point anywhere, and 469 instances of the OR set a point better than every published one.
- In total 648 (41%) of the 1,594 instances are settled, 608 of them optimal: 528 by published work and 120 (7.5%) by this work. All results are released with an updated status file in MIPLIB's `.solu` format.

What is new, and what is reused. New are the reconciliation itself, the robust subset-sum search with tables rebuilt at every node, the use of Brouwer's stability theorem for edge clique interdiction, the Lean proofs and the verified data package. The other arguments are known and are applied here systematically: dualizing the follower's linear programming relaxation into one mixed-integer program[20–22] and the integrality gap of a packing problem with few rows, which follows from the fact that a basic LP solution has at most as many fractional variables as rows;[23] the online lower bound of Weninger and Fukasawa;[17] Meyer's argument that a rational mixed-integer program that is feasible and bounded attains its optimum;[24] the bound HPR $\leq$ OPT of the high point relaxation;[25] and exhaustive enumeration.

Section 2 recalls the problem class and the families in BOBILib. Section 3 describes the methods and how each claim is checked, Section 4 gives the computational setup and the results, and Section 5 discusses why general solvers miss these results, what a benchmark library could do about it, the limitations of this work and what stays hard. The proofs, the verification protocol and the per-instance tables are in a separate supplementary document; its sections are cited as Supp. Sec. A, B, and so on, and its results as, for example, Supp. Lemma B.1.

2. BACKGROUND

2.1 Bilevel Problems in BOBILib Form

BOBILib writes every instance with a minimizing leader:

$$\min_{x,y} \; c^\top x + d^\top y \quad \text{s.t.} \quad Ax + By \geq a, \qquad y \in \arg\min_{y'} \big\{ f^\top y' : Cx + Dy' \geq b \big\}, \tag{1}$$

with integrality requirements on some of the leader's variables x and the follower's variables y. For a fixed x the follower solves $P(x) = \min\{f^\top y : Cx + Dy \geq b\}$ (with its integrality requirements); x enters it only through the right-hand side. A point (x, y) is *bilevel feasible* if it satisfies the leader's rows and integrality requirements and y is optimal for $P(x)$. When the follower has several optimal answers, the one best for the leader is assumed (the optimistic convention, which BOBILib uses throughout). An upper-level row with a nonzero coefficient on a follower variable is a *coupling row*; a leader variable that appears in the follower's rows is a *linking variable*. The *high point relaxation* (HPR) drops the follower's optimality and keeps every row of both levels,

$$\text{HPR} = \min\big\{ c^\top x + d^\top y : Ax + By \geq a,\ Cx + Dy \geq b,\ \text{integrality} \big\},$$

so HPR $\leq$ OPT, the bilevel optimum.

2.2 The Families of BOBILib

Most of BOBILib's unsolved instances are interdiction games: the leader removes parts of the follower's problem within a budget, and the two objectives are opposite, so the game is zero-sum and the optimistic and pessimistic readings agree. Table 1 lists the families with their outcomes.

Packing interdiction. Items are $N = \{1, \ldots, n\}$. The leader chooses $x \in \{0,1\}^n$ within budget rows $Gx \leq g$, and the follower packs what is left:

$$\Phi(x) = \max\big\{ p^\top y : Wy \leq c,\ y \leq \mathbf{1} - x,\ y \in \{0,1\}^n \big\}, \qquad \text{OPT} = \min\big\{ \Phi(x) : Gx \leq g,\ x \in \{0,1\}^n \big\}, \tag{2}$$

with nonnegative integer profits p, weights W and capacities c (from here on, c denotes capacities, not the cost vector of (1)). The knapsack interdiction problem (KIP) has one leader row, with budget β, and one follower row, with capacity c; the multidimensional KIP (MKIP) has several of either. In the *subset-sum class* of KIP each item's profit equals its weight ($p = w$), so the follower's value is the largest sum of surviving weights that fits the capacity. Figure 3 shows a small instance. BOBILib's KIP instances come from DeNegre,[26] whose set is built from bicriteria instances of the Multiple Criteria Decision Making library, and from Caprara, Carvalho, Lodi and Woeginger[20] and Fischetti, Monaci and Sinnl,[27] whose sets use the item classes of the knapsack generator of Martello, Pisinger and Toth.[28] Its MKIP instances form two sets of Fischetti, Ljubić, Monaci and Sinnl,[18] the 144 *imkp* instances built on the SAC-94 suite[29] and the 810 *OR* instances built on Chu and Beasley's multidimensional knapsacks from the OR-Library (Operations Research Library).[30,31] The OR instances have 100 to 500 items and a tightness $\alpha \in \{0.25, 0.5, 0.75\}$: the right-hand side of every knapsack row, the

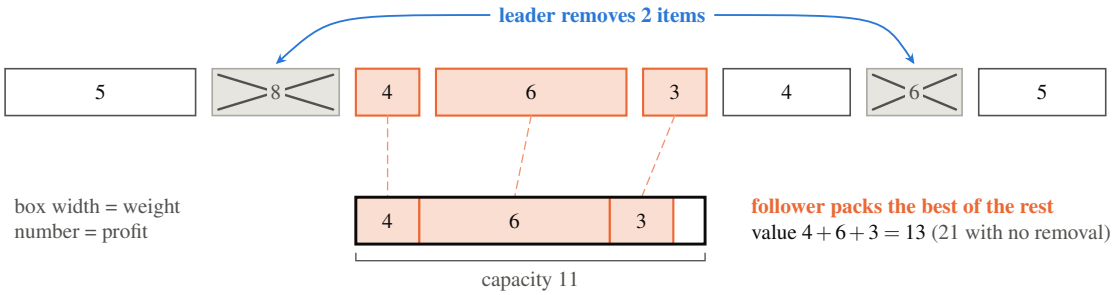


Figure 3. Knapsack interdiction on a toy instance (box width = weight, number = profit). The leader removes two items, and the follower packs the best of what is left into its knapsack. The removal shown cuts the follower's best packing from 21 to 13, and no other pair of removals does as well.

Table 1. BOBILib's families and the outcome for the instances it lists as feasible or open. Published: optimal in published work (for 82 of them only the published value is used; Section 3.1). Here: proven optimal in this work. No point: no bilevel feasible point exists. New point: a verified bilevel feasible point where BOBILib lists none (for 62 of them a point had been published; Section 4.2). The total includes BOBILib's 24 assignment interdiction instances, which it lists as optimal.

		Feasible	Outcome for the feasible or open instances				
Family	Instances	or open	Published	Here	No point	New point	Unchanged
Knapsack interdiction (KIP)	599	392	376	16	0	0	0
Multidim. knapsack interdiction (MKIP)	954	862	140[†]	0	0	140	582
Edge clique interdiction	220	60	0	57	0	0	3
Network interdiction	72	2	0	2	0	0	0
Generalized interdiction	90	12	4	0	0	0	8
General, pure integer	359	105	8	0	6	15	76
General, mixed integer	336	161	0	5	34	75	47
Total	2,654	1,594	528	80	40	230	716

[†]Includes OR10x100-0.75_4.dat-0, whose published lower bound is inconsistent but whose published branch-and-cut bound closes it (Section 4.4).

leader's budget rows and the follower's capacity rows alike, is α times the sum of its coefficients (below, "budget 25%" means $\alpha = 0.25$). They come in three variants: variant `-100` has one follower knapsack, variants `-0` and `-50` have several.

Clique interdiction. The leader deletes a set D of at most k edges of a graph $G = (V, E)$ with n vertices and m edges, and the follower takes a maximum clique of $G - D$, of size $\omega(G - D)$. BOBILib's *clique* set of Fischetti, Monaci and Sinnl[27] has 60 instances with $n \in \{40, 50, 60\}$, edge density 0.7 or 0.9 and $k = \lfloor m/4 \rfloor$; its other clique instances use the graphs of Tang, Richard and Smith.[32]

Network interdiction. In the attack-protect game of Baggio, Carvalho, Lodi and Tramontani[33] the leader attacks at most Φ nodes, the follower then protects at most Λ nodes, and the attack spreads from attacked nodes through unprotected neighbors. The follower maximizes the number of saved nodes and the leader minimizes it. (Here Φ is a budget, as in BOBILib's row names, not the value function of (2).)

General instances. BOBILib builds most of its general instances from MIPLIB problems, by assigning the variables and rows to the two levels and keeping the problem's objective as the leader's.[34–36] Other general sets come from DeNegre,[26] Xu and Wang,[37] Ozaltin and Zhang[38] and Fischetti et al.[15] Almost all of the 266 unsolved general instances are MIPLIB-derived; BOBILib lists another 33 as infeasible. Generalized interdiction[27] and assignment interdiction are smaller families.

2.3 What the Literature Had Solved

Knapsack interdiction. Caprara, Carvalho, Lodi and Woeginger[20] gave an exact algorithm for KIP, and Della Croce and Scatamacchia[39] improved on it. Weninger and Fukasawa's combinatorial branch and bound[17,40] is the state of the art: it solves 583 of BOBILib's 599 KIP instances, 376 of which BOBILib does not list as optimal. The 16 instances it leaves open, in runs with 16 threads and a one-hour limit, have 200 to 500 items, and in all of them each profit equals its weight:

8 are in the subset-sum class of the generator (class 6) and 8 in its even-odd class (class 7: even weights, odd capacity). Their reimplementation of Della Croce and Scatamacchia's algorithm and two-hour runs of Lozano, Bergman and Cire's decision diagrams[19,41] leave them open too; Weninger and Fukasawa's published run records[42] give lower bounds 2 to 8 units below BOBILib's values. In every one of these instances BOBILib's value equals the follower's capacity c (class 6) or $c-1$ (class 7, where every subset sum is even and c is odd), a trivial upper bound on the follower's value under any leader choice. So the open question was only the lower bound: can the leader keep the follower from filling its knapsack exactly (up to parity)?

Multidimensional knapsack interdiction. Fischetti et al. solved all 144 imkp instances with interdiction cuts;[18] BOBILib lists 52 of them as only feasible, with values up to 23% above the optimum. No OR instance is optimal in BOBILib. Lozano, Bergman and Cire[19] proved 88 of them optimal and published lower bounds and points for most of the others,[41] and Lefebvre and Schmidt[43] published further points; 139 OR instances had no published point.

Clique, network and general instances. Furini et al. gave exact algorithms for vertex and edge clique interdiction,[44,45] but the 60 instances of the clique set had no published optimum. Deciding whether at most k deletions bring the clique number down to a given value is the edge blocker clique problem.[46] For general MIBLPs the methods are branch and bound[25] and branch and cut,[13,47] intersection cuts,[15,34,35] value-function reformulations,[48,49] decision diagrams,[19,50] a Dantzig–Wolfe single-level reformulation[43] and heuristics such as a penalty alternating direction method (PADM).[36] Published work solves 8 MIPLIB-derived instances that BOBILib does not list as optimal: 5 by Lozano et al.; air03-0.100000 by Fischetti et al.;[15] p0282-0.500000 by Vásquez, Lozano and van Hoeve;[50] and neos-950242_0_100, which an earlier BOBILib release listed as optimal and whose BOBILib point also meets the MIPLIB bound of Section 3.6 (Supp. Table S11). Lozano et al. also solve 4 generalized interdiction instances.

3. METHODS

3.1 Reconciling the Statuses and Verifying Every Claim

Published results. For every family I compared BOBILib's status and value with the per-instance data the authors publish: Weninger and Fukasawa's answer files and run records,[42] the tables of Fischetti et al., the solution files of Lozano, Bergman and Cire,[41] the run files of Lefebvre and Schmidt,[43] and the other papers cited above. An instance counts as solved in published work only when a proven optimal value is published for it. Each KIP instance was matched to Weninger and Fukasawa's file by its capacities and the multiset of item triples (profit, follower weight, leader cost); every match is unique. For 446 of the 528 published optima a verified solution file at the published optimum is in the data package: for all 376 KIP instances it was rebuilt from Weninger and Fukasawa's leader choices, and for 70 of the 88 OR instances this work's own point has the published value. For the other 82 (52 imkp, 18 OR, 8 MIPLIB-derived and 4 generalized interdiction instances) only the published value is used. At 335 of the KIP instances BOBILib's value is above the optimum, by a median of 8% and up to 672%. The literature's optimality proofs are not repeated here.

Verification. Every solution file of this work uses BOBILib's format and passes `bobi verify`, a verifier written for this work after BOBILib's own feasibility checker (Supp. Sec. A.2). It is separate from BOBILib's solvers, but not from the rest of this work's code: it shares the instance reader and the exact knapsack oracle with the solvers. It checks (a) every row, bound and integrality requirement of the HPR at the file's point; (b) the stated leader objective; and (c) follower optimality: with the leader's variables fixed, the follower's problem is solved again, with one follower row by an exact knapsack oracle, a branch and bound with Dantzig's linear programming (LP) bound that falls back to dynamic programming,[51,52] and otherwise as a mixed-integer linear program (MILP) with HiGHS.[53] The file's follower value φ_{sol} must pass BOBILib's relative test against the proven optimum φ_{check},

$$\frac{|\varphi_{\text{sol}} - \varphi_{\text{check}}|}{10^{-10} + |\varphi_{\text{check}}|} \leq 10^{-4}. \tag{3}$$

Table 2 collects the tolerances used here and by BOBILib's checker. Code that shares nothing with the solvers re-checks the knapsack interdiction lower bounds, the recession directions and the HPR comparisons (Section 3.7). In this paper "optimal" means a bilevel feasible point in this sense whose value equals a proven lower bound, or, for the 82 published optima without a solution file here, the published claim. Each lower bound of this work is a theorem with a published certificate, an exact search, an exhaustive enumeration or, for one OR instance, a published bound, as described below. The full protocol is in Supp. Sec. A.3.

Table 2. Tolerances of the checks.

Check	Tolerance
`bobi verify`: rows, bounds and integrality of the HPR at a file's point	10^{-6} absolute
`bobi verify`: follower optimality, test (3) (differences up to 10^{-9} count as zero)	10^{-4} relative
`bobi certify` (points with coupling rows): follower optimality and optimistic tie-break, by proven dual bounds	10^{-5} relative
Best response inside `bobi general` and `bobi coupled`: follower value above its optimum	$10^{-7}(1+\|\varphi\|)$
BOBILib's checker: rows and integrality (MIPLIB solution checker)	10^{-4}
BOBILib's checker: follower optimality, test (3)	10^{-4} relative

3.2 Knapsack Interdiction: A Robust Subset-Sum Search

All 16 KIP instances open in the literature are subset-sum instances: the follower's value is the largest sum of surviving weights that fits its capacity. The search answers a yes/no question: given an integer ℓ, does every leader choice within budget leave the follower a subset sum in the band $[\ell, c]$? A "yes" proves OPT $\geq \ell$; a "no" comes with a leader choice whose follower value is below ℓ. The loop starts from Weninger and Fukasawa's greedy leader choice and asks the question with ℓ equal to the value of the best leader choice found so far; it stops at the first "yes", which proves optimality (Supp. Cor. B.25). For each of the 16 instances it stopped at BOBILib's value, c or $c-1$, so the band $[\ell, c]$ has width 0 or 1. The rules below hold for any band; for an instance whose optimum lies far below c the band is wider, and the loop may need several searches.

Count formulation. Subset-sum instances have many items with the same weight. Such items are interchangeable for the follower, so the leader should delete the cheapest copies first, and the search branches on how many copies of each weight survive rather than on which items are deleted (Figure 4a; Supp. Prop. B.4, which also divides the weights by their greatest common divisor). This removes many symmetric branches.

Lemma 1 (Monotonicity and cheapest copies)**.** *(i) Blocking more items never helps the follower: $X \subseteq X'$ implies $\Phi(X') \leq \Phi(X)$. (ii) For a class Q of identical items and any leader set X there is a leader set X' that agrees with X outside Q, blocks as many copies of Q, blocks only copies no costlier than every copy it leaves, costs no more than X, and has $\Phi(X') = \Phi(X)$.*

Proof in Supp. Lemmas B.1 and B.2; Lean: `follower_antitone`, `cheapest_first`.

Windows. At a node of the search the decided classes have left survivors M whose subset sums are known. A run of consecutive reachable sums only grows when a copy of value v is added, as long as v is at most the run's width (Figure 4b; Lean: `window_extend`, `window_extend_all`). If the leader cannot afford to delete enough weight to stop such a window before it reaches the band, the node is safe whatever the leader does (Supp. Prop. B.8).

The online game. The strongest rule is an online relaxation due to Weninger and Fukasawa. Fix an order of the items. In the online game the leader decides item by item whether to block it, and the follower, who has seen every earlier move, decides whether to take each unblocked item. The leader may react to the follower's earlier choices, which it cannot do in the real game, so the follower's value in the online game is a lower bound:

Theorem 2 (Online lower bound[17])**.** *For every list o of distinct items, the value $\Omega(o, \beta, c)$ of the online game, computed by backward induction, satisfies $\Omega(o, \beta, c) \leq \Phi(X)$ for every leader set X within the budget β. Hence $\Omega(o, \beta, c) \leq$ OPT.*

Proof in Supp. Thm B.9; Lean: `online_le_K_any`, `online_lower_bound`.

Tables with an end set. For subset-sum instances I play the online game class by class, in the count formulation and with a target. At each class the leader announces how many copies it deletes; the follower then takes some of the surviving copies, which lowers a residual t; the follower wins if the final residual lies in an *end set* A. Let $G_A(Q, \beta')$ be the set of residuals from which the follower wins when the classes Q are played in a given order and the leader has budget β'. Then every deletion within β' leaves a subset sum s with $t - s \in A$ (Supp. Thm B.12; Lean: `GA_sound`, and `GA_sound_perm` for any order of play). A follower who wins against a budget also wins against any smaller one, so the table is stored as one number per residual: the largest budget against which the follower still wins. These thresholds satisfy a min-max recursion over the classes, and one backward pass computes them for all residuals (Supp. Thm B.17; Lean: `thr_iff`, `GA_iff_thr`).

The per-node rule. The decided survivors at a node already reach a set S of subset sums, and the follower wins with any total in $[\ell, c]$. So the node's end set is $A_{\text{node}} = \{a \in [0, c] : s \leq a \leq s + (c - \ell) \text{ for some } s \in S\}$, and the game starts from the residual c: if it ends at $a \in A_{\text{node}}$, the follower's total $c - a$ plus a suitable $s \in S$ lies in the band.

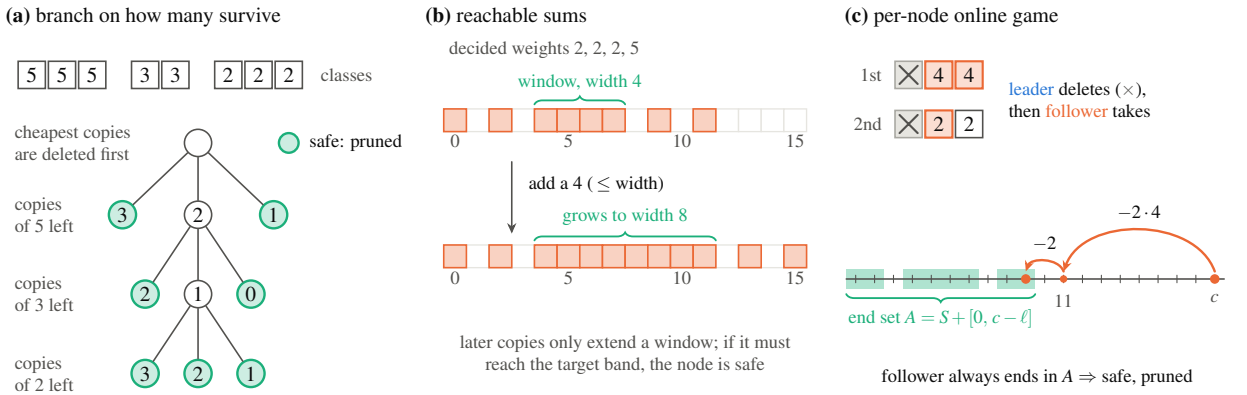


Figure 4. The search that proves the 16 new knapsack interdiction optima, on toy numbers. (a) It branches on how many copies of each weight survive. (b) A window of consecutive reachable sums only grows as copies are added. (c) At each node an online game, in which the leader deletes one weight class at a time, decides whether the follower can always end in the set A; if so the node is pruned.

Theorem 3 (Per-node rule)**.** *At a node with decided sums S and residual budget β', let π be any order of the undecided classes. If β' is at most the threshold of the residual c in the table for π with end set A_{node}, then every completion of the node within β' leaves a subset sum in $[\ell, c]$: the node can be pruned, and it stays safe for every smaller budget.*

Proof in Supp. Thm B.19; Lean: `node_table_sound_perm` and `node_table_threshold_sound_counts`.

Figure 4c shows the rule on toy numbers. The decided sums are $S = \{0,3,5,8\}$ and the band is $[18,19]$, so $A_{\text{node}} = \{0,1,3,4,5,6,8,9\}$. Three copies of 4 and three of 2 are undecided, each copy costs 1 to delete, and the leader has budget 2. The game starts at the residual 19. In the play shown, the leader deletes a 4 and the follower takes the other two (residual 11); the leader deletes a 2 and the follower takes one of the two left (residual $9 \in A_{\text{node}}$). The follower's total $8+2=10$ plus $s=8$ is 18, in the band. Checking every play shows that the follower can always end in A_{node} at budget 2, so the node is pruned.

The table is rebuilt from the node's own sums at every node that would otherwise branch, and its threshold is cached, so later nodes with the same depth and sums and a budget below the threshold are pruned at once. The search also uses cheaper rules: the *direct* rule (the decided sums S already meet $[\ell, c]$), guaranteed copies, the window rule and the cache (Supp. Thm B.23). The order of play does not affect soundness but matters a great deal for strength. The search plays the classes that are cheapest to delete per unit of weight first: the follower then sees the leader's likely deletions before it commits, and keeps the expensive classes for fine adjustment. On interKP-200-100-6-2, tables built only at the root needed 1,128,566 nodes when played largest weight first and 513,834 when played cheapest first; per-node tables need 303. A node is declared "blocking" only at a leaf no rule prunes, which yields a leader choice below ℓ; so a search that ends within its node limit is exact (Supp. Thm B.24).

3.3 Multidimensional Knapsack Interdiction: A Linear Programming Bracket

Let $U(x)$ be the value of the follower's LP relaxation in (2), in which the follower may take fractions of items, and $p_{\max}$ the largest profit. Dualizing the follower's LP gives

$$U(x) = \min_{\lambda \geq 0} \; c^{\top}\lambda + \sum_{j=1}^{n} \max\bigl(0,\, p_j - (W^{\top}\lambda)_j\bigr)(1 - x_j),$$

and for binary x the product linearizes exactly, so $U^* = \min\{U(x) : Gx \leq g\}$ is the value of a single MILP (Supp. Prop. C.2). Caprara et al.'s algorithm for KIP rests on this dualization,[20,39] and it is the classical single-level reformulation of network interdiction.[21,22] An optimal vertex of the follower's LP has at most m fractional items when the follower has m rows, which bounds the gap between U and Φ (Figure 5).

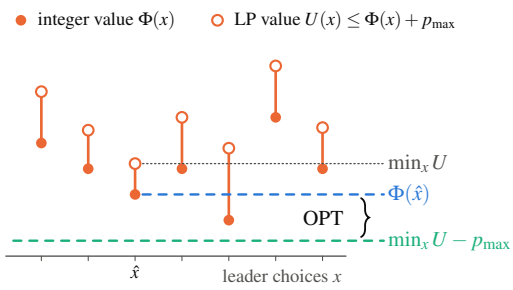


Figure 5. The LP bracket for one follower knapsack (made-up numbers): the follower's LP value $U(x)$ exceeds its integer value $\Phi(x)$ by at most $p_{\max}$, so the optimum lies between $\min_x U - p_{\max}$ and $\Phi(\hat{x})$, where $\hat{x}$ minimizes U.

Theorem 4 (LP bracket)**.** *With m follower rows, $U(x) - m\,p_{\max} \le \Phi(x) \le U(x)$ for every leader choice x. Hence for any lower bound $L \le U^*$, for example a MILP solver's proven dual bound, and any leader-feasible $\hat{x}$: $\lceil L - m\,p_{\max} \rceil \le \mathrm{OPT} \le \Phi(\hat{x}) \le U(\hat{x})$.*

Proof in Supp. Lemma C.3 and Thm C.4; not formalized.

With one follower knapsack (variant `-100`, $m = 1$) the bracket is at most one item's profit wide when the MILP is solved to optimality ($L = U^*$ at its minimizer $\hat{x}$); with a time limit its width is $\Phi(\hat{x}) - \lceil L - p_{\max} \rceil$. A second lower bound extends Theorem 2 to several leader rows: merge them into one row with nonnegative multipliers, then round the leader's costs down and its budget up, and the follower's weights up and its capacity down. Each step only helps the leader, so the online game on the merged data is still a lower bound (Supp. Thm C.5; the online bound is formalized, the merging and rounding are not). Upper bounds come from a first-improvement local search on the leader's choice that evaluates each candidate with the exact knapsack oracle, so every point it reports is certified.

With several follower knapsacks (variants `-0` and `-50`) the bracket's width $m\,p_{\max}$ is too large for a useful lower bound, and only the literature's lower bounds are used. The minimizer $\hat{x}$ of the MILP is still a good leader choice: whenever HiGHS solves the follower's integer problem at $\hat{x}$ to proven optimality, the point is certified (Supp. Sec. C.4).

3.4 Clique Interdiction: Extremal Graph Theory and Partitions

After any deletion at least $m - k$ edges remain, and extremal graph theory says how many edges force a clique of a given size.

Theorem 5 (Clique bounds)**.** *Let G have n vertices and m edges, and let the leader delete at most k edges.*

(i) (Mantel[54]) If $m - k > \lfloor n^2/4 \rfloor$, every remainder contains a triangle, so the value is at least 3.

(ii) (Partitions) If the vertices can be split into r parts with at most k edges inside the parts, deleting those edges leaves an r-partite graph, so the value is at most r.

(iii) (Brouwer[55, 56]) If $n \ge 7$, $m - k \ge t_3(n) - \lfloor n/3 \rfloor + 2$, and every split of the vertices into 3 parts has more than k edges inside the parts, every remainder contains a K_4, so the value is at least 4.

Proof in Supp. Thms D.2 and D.6 and Prop. D.3. Parts (i) and (ii) are proved in Lean (`mantel`, `triangle_survives` and `cliqueFree_of_partition`); part (iii) is not formalized, as it rests on Brouwer's theorem.

Here $t_3(n)$ is the number of edges of the balanced complete 3-partite graph on n vertices, the Turán number.[57] Brouwer's Theorem 2 of 1981, in an unrefereed report,[55] says that a K_4-free graph on $n \ge 7$ vertices with at least $t_3(n) - \lfloor n/3 \rfloor + 2$ edges is 3-partite. Tyomkyn and Uzzell give a new proof of this statement in a refereed journal (their Theorem 1.1, for K_{r+1}-free graphs; here $r = 3$).[56] It was also found again by Hanson and Toft and by Kang and Pikhurko,[58,59] and Amin, Faudree, Gould and Sidorowicz determine the largest number of edges of a maximal K_p-free graph that is not $(p-1)$-partite, which for $p = 4$ gives the same threshold.[60] So in (iii) a K_4-free remainder would be 3-partite, the deletion would contain every edge inside the parts of some 3-partition, and that takes more than k deletions. Table 3 gives the numbers. At density 0.7 Mantel's theorem applies. At density 0.9 Brouwer's threshold is met, while $m - k$ stays below $t_3(n)$, so Turán's theorem alone would not give a K_4. Figure 6 shows the argument on a small graph.

Table 3. Edge counts behind the clique lower bounds. Every instance of a group has the same n, m and k. Mantel's theorem needs $m-k > \lfloor n^2/4 \rfloor$; Brouwer's needs $m-k \geq t_3(n) - \lfloor n/3 \rfloor + 2$.

n	Density	m	k	$m-k$	$\lfloor n^2/4 \rfloor$	$t_3(n)$	$t_3(n) - \lfloor n/3 \rfloor + 2$	Optimum
40	0.7	546	136	410	400	533	522	3
50	0.7	857	214	643	625	833	819	3
60	0.7	1,239	309	930	900	1,200	1,182	3*
40	0.9	702	175	527	400	533	522	4
50	0.9	1,102	275	827	625	833	819	4
60	0.9	1,593	398	1,195	900	1,200	1,182	4

* Except three instances of this group, which are open, with value 3 or 4 (Section 4.5).

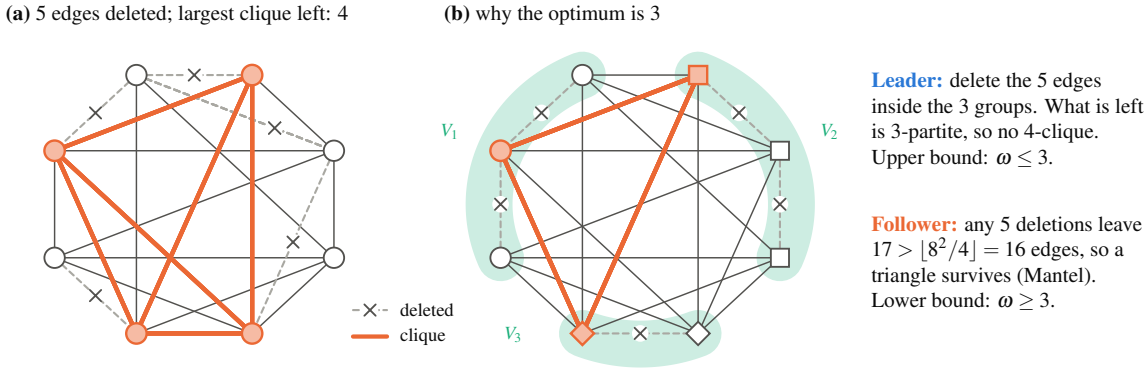


Figure 6. Clique interdiction on an 8-vertex graph with budget 5. (a) A deletion that still leaves a 4-clique. (b) Why the optimum is 3: deleting the 5 edges inside the groups V_1 to V_3 leaves a 3-partite graph, so no 4-clique survives (Theorem 5(ii)), and any 5 deletions leave more than $\lfloor 8^2/4 \rfloor$ edges, so a triangle survives (Theorem 5(i)).

The last condition of (iii) reduces to finitely many cases. Counting the non-edges inside the parts, a 3-partition with given part sizes has few edges inside its parts exactly when few edges of the sparse complement graph join different parts. Only a handful of part-size vectors leave any room, and for each of them a partition with at most k inside edges would need at most a few complement edges between the parts (Supp. Prop. D.7). That gives 170 cases (instance, part sizes), each decided infeasible by several methods: CP-SAT[61] and SCIP[62,63] on two different models, an exact depth-first search that uses no solver, and, in a separate review session with its own code, HiGHS on a third model and a branch and bound written in C. These verdicts are computations, not formal proofs. For the upper bounds, a tabu search looks for 3- and 4-partitions with few inside edges. Each solution file deletes exactly the inside edges of a partition and states a maximum clique of the remainder, computed exactly and checked again with two exact clique searches.[64–66]

3.5 Network Interdiction: Every Attack, with an Exact Tree Program

For a fixed attack, the follower's best response in the attack-protect game is a closure problem: the burned nodes are the smallest set that contains the attacked nodes and is closed under spreading to unprotected neighbors (Supp. Lemma F.1). On a forest it is solved exactly by a dynamic program (DP) over the states (protected, saved) of each node, with the number of protected nodes as a knapsack dimension (Supp. Sec. F.3). The two instances BOBILib lists as feasible are forests with 49 nodes and $\Phi = \Lambda = 5$, so all 2,138,410 attacks of at most Φ nodes can be enumerated (Figure 7). Every follower point is checked against the model as read from the files, and at the minimizing attack the follower's MILP is solved again by HiGHS. The minimum over all attacks is then the optimum (Supp. Thm F.4).

3.6 General Instances: Recession Directions, the MIPLIB Bound and Feasible Points

No bilevel feasible point. The leader's variables enter the follower's rows only as constants. So a direction along which the follower can improve without limit works for every leader choice at once (Figure 8).

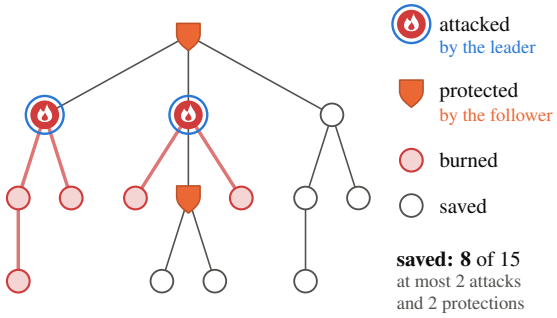


Figure 7. The attack-protect game on a toy tree: the leader attacks, the follower then protects, and the attack spreads through unprotected neighbors. Against the optimal attack shown, the follower saves at most the nodes marked as saved.

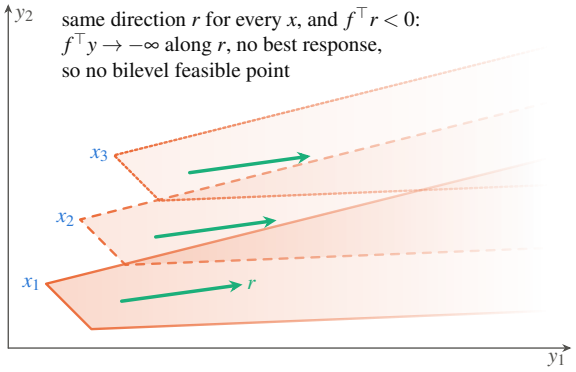


Figure 8. Why some MIPLIB-derived instances have no bilevel feasible point: the follower's region moves with the leader's choice x but keeps the same direction r, and $f^\top r < 0$ leaves the follower without a best response for every x.

Theorem 6 (Recession direction)**.** *Let r be a rational direction of the follower's variables that keeps every follower row and bound satisfied when added to a feasible point ($D_i r \geq 0$ for a row with a finite lower side, $D_i r \leq 0$ for a row with a finite upper side, and the same sign conditions for the variable bounds), and let $f^\top r < 0$. Then for every leader choice the follower's problem is infeasible or unbounded, so the bilevel problem has no feasible point.*

Proof in Supp. Thm E.1, after Meyer;[24] not formalized.

The integrality of the follower is no obstacle because r is rational: a multiple of r keeps the integer variables integral. The directions are found by an LP solver in floating point. A script that reads the instance files independently of the solver code converts each direction to exact rationals and checks the conditions of Theorem 6 in rational arithmetic.

The MIPLIB bound. Every bilevel feasible point is feasible for the HPR, so HPR $\leq$ OPT, and a bilevel feasible point with value HPR is optimal (Supp. Lemma E.3). When BOBILib builds an instance from a MIPLIB problem and changes nothing else, the HPR *is* that MIPLIB problem, and its published optimum[10] is a lower bound on the bilevel optimum. This is checked, not assumed: a script compares the bilevel instance with the MIPLIB 2017 model in exact rational arithmetic (variables, rows, coefficients, sides, bounds, integrality and objective).

Feasible points. Without coupling rows, a bilevel feasible point follows from any HPR point at which the follower's problem has an optimal solution: fix the leader's part, solve the follower to optimality, and choose among the follower's optima the one best for the leader, the optimistic tie-break (Supp. Prop. E.4). Unlike BOBILib's two solvers, this handles continuous linking variables. With coupling rows, all follower optima at the HPR's leader values may violate a coupling row. For fixed values of the linking variables at which the follower's problem has an optimum, two solves, the follower's problem and then one MILP for the optimistic best response with the coupling rows, find the best bilevel feasible point with those values or show that there is none (Supp. Prop. E.5). A portfolio of heuristics, `bobi coupled`, searches for usable linking values: the HPR's own values, best-response dynamics, an HPR whose objective is penalized by the follower's, PADM,[36] moves of continuous linking variables into the interior, value-function cuts[48,49] and no-good cuts. None of these is needed for correctness. What is certified is each returned point, checked again from scratch with proven dual bounds on the follower's problem and on the optimistic tie-break (Table 2), and then by `bobi verify`.

3.7 Formal Verification and Separate Checks

The main lemmas behind the new exact search and the clique bounds are proved in the Lean 4 theorem prover[67] with its mathematical library Mathlib[68] (Table 4). The project contains no `sorry`, and every main theorem uses only Lean's standard axioms (`propext`, `Classical.choice` and `Quot.sound`). The Lean statements are about abstract models (items, weight classes, graphs), not about the Rust code: that the code computes the formalized recursions is argued in its documentation and tested against brute force, not proved. The full map from results to Lean theorems and to the code that relies on them is in Supp. Sec. G.

Table 4. What is machine-checked. Lean theorem names are in the namespace `Bobi` of the project `bobilib/lean`; Supp.: section of the supplementary document.

Result	Lean theorems	Supp.
Monotonicity, cheapest copies first (Lemma 1)	`follower_antitone`, `cheapest_first`	B.1
Windows of reachable sums	`window_extend(_all)`, `window_survivors`	B.2
Online lower bound (Theorem 2)	`online_lower_bound`, `online_le_K_any`	B.3
Tables with an end set, any order of play	`H_sound(_any)`, `G_sound`, `GA_sound`, `GA_sound_perm`, `covers_sound`, `band_covers_sound`	B.4
Threshold form	`thr_iff`, `GA_iff_thr`, `covers_threshold_sound(_perm)`, `band_covers_threshold_sound(_perm)`	B.5
Per-node rule (Theorem 3)	`node_table_sound`, `node_table_sound_perm`, `node_table_threshold_sound_counts`	B.6
Mantel's theorem, deletion form (Theorem 5(i))	`mantel`, `triangle_survives`	D.2
Partition upper bound (Theorem 5(ii))	`cliqueFree_of_partition`	D.3

Not formalized: the search code itself; the reduction to the count formulation with its scaling by the greatest common divisor (Supp. Prop. B.4); the fractional deletion bound in the window rule; the direct, guaranteed-copies and cache rules and the node-state lemma (Supp. Thm B.23 and Lemma B.22); restricted copies (an option no recorded run uses); the LP bracket and the merged-row bound; Brouwer's theorem and the part-size verdicts; the network DP; the recession directions and the MIPLIB comparison.

Separate checks. The software, the experiments and the checks below were carried out with an AI coding agent directed by the author (see the Acknowledgements). The checks described as separate reviews ran in fresh sessions of the same agent that wrote their own code and their own instance readers, without reading the code they checked; they are separate re-implementations, not independent human reviews. The 16 subset-sum lower bounds were proved again by two Python provers that share no code with the Rust search: one written with the search, the other in a separate review session. Both parse the instance files themselves and use only the direct rule and the per-node rule, in integer arithmetic. Two other designs of the search, built separately, prove interKP-200-100-6-3, and every search binary was compared with exhaustive enumeration on 2,990 random small instances, with 0 mismatches. The clique, network and coupled-row results were each derived again in a separate review session with its own instance reader and different solvers or code: two exact maximum-clique searches, HiGHS and a branch and bound in C for the part-size cases; a C implementation of the tree DP compared with ours at every attack; and a second reading of the coupled files with HiGHS's own reader. The reviews' notes and their corrections are in the `README` files of the data package; Section 5.2 discusses the risk of errors that all these checks share.

4. RESULTS

4.1 Computational Setup

The computations ran on a virtual machine with four cores of an Intel Xeon processor at 2.1 GHz and 15 GB of memory (Ubuntu 24.04; Rust 1.97; Python 3.13; HiGHS 1.15, SCIP 10.0 through PySCIPOpt 6.3, CP-SAT 9.15; Lean 4.35 with Mathlib). The machine was shared with other jobs and often oversubscribed. Every method ran on one thread, except CP-SAT, which used two workers. Table 5 lists the runs. An unsolved instance outside them was not attempted; in particular the general instances that BOBILib lists as feasible were only compared with the literature and, where their point meets the MIPLIB optimum, with the MIPLIB bound. Times are CPU seconds where so stated and wall-clock seconds otherwise, and no proof depends on them. The total computing effort was not recorded; the data package gives per-instance times where measured.

4.2 Overview

Table 1 shows the outcome by family. All 392 unsolved KIP instances are now optimal. Of the 140 MKIP instances that get a verified point where BOBILib lists none, 88 get the first point anywhere; for the other 52 Lozano, Bergman and Cire had published a point with a solution file. The 582 unchanged MKIP instances include many with better points

Table 5. Methods, the instances they were run on, their limits per instance, and their outcomes (Supp. Sec. H and the `README` files of the data package give the commands).

Method	Run on	Limits	Outcome
Robust subset-sum search	the 16 open KIP instances; 36 with published optima	none reached; every run finished	16 optimal; all 36 published optima reproduced
LP bracket, merged-row bound, local search	270 OR, one follower knapsack	MILP 60 s, local search 30 s	270 points, 191 better lower bounds
Follower LP at the bracket's minimizer	540 OR, several follower knapsacks	MILP 30 s, follower 60 s	420 points
Partitions and part-size cases	60 clique	part-size cases run until decided	57 optimal, 3 open
Enumeration with tree DP	2 feasible and 34 optimal trees	complete; 80 s and 70 s for the two	2 optimal; 34 values reproduced
Recession directions	general instances	LP, then exact check	40 without a feasible point
HPR, follower, tie-break (`bobi general`)	the 98 open ones without coupling rows	HPR 300 s, follower 300 s	64 points; 17 rays, 10 without an HPR point, 6 follower unsolved, 1 failed check
Portfolio (`bobi coupled`)	54 of the 87 open ones with coupling rows (23 rays and 10 with over 100,000 variables left out)	600 s or 300 s, follower 120 s, batch budget 2.5 h	52 attempted: 29 points, 23 none

now (Section 4.4). Of the 90 general instances that get a verified point, 80 get the first point anywhere. Of the 80 new optimality proofs, 20 confirm a value BOBILib already listed (the 16 KIP instances, the 2 network instances and the 2 noswot instances): there the proof is new, not the value.

4.3 Knapsack Interdiction

All 16 KIP instances without a published optimum are proven optimal (Table 6). In every case BOBILib's value was already optimal: it equals the capacity c, or $c-1$ in class 7, and every leader choice within budget attains it. The new result is the lower bound. The 12 instances other than those of class 6 with budget index 3 take at most 651 nodes and 0.26 seconds. Those of class 6 with budget index 3 are the hardest, with up to 19,405 nodes; the slowest, interKP-500-100-6-3 (optimum 6,976), takes 10,079 nodes and 10.0 seconds. All 16 together take 32.5 seconds. As a check on instances with known answers, the search and the prover written with it reproduce Weninger and Fukasawa's optimum on all 36 class 6 and 7 instances with 100 or 200 items that have one, most of them with optima far below c.

Figure 9 compares the search before and after the per-node tables; "before" is the version immediately preceding the per-node tables, run on the same machine. On interKP-200-100-6-2 it needed 3,926,223 nodes and 22.8 seconds, against 303 nodes and 0.10 seconds now, and it did not finish interKP-200-100-6-3 in a run of 4 hours. The second prover, which uses only the direct and per-node rules, needs almost the same number of nodes. Weninger and Fukasawa's solver was not run again here: it needs Gurobi, which is not available on this machine, and their published runs (16 threads, one hour) already leave these instances open. The times are therefore not a like-for-like comparison with their solver.

4.4 The OR Set

Table 7 and Figure 10 show the relative optimality gap $(U-L)/U$, where L and U are the best lower and upper bounds from the literature and this work (the upper bound may be an unverified published point), over the instances with both a published lower and a published upper bound. The published points come from Lozano, Bergman and Cire[19,41] and Lefebvre and Schmidt.[43] At 4 instances a published lower bound exceeds a published point; these are left out of the medians and listed in Supp. Table S6. For three of them, OR5x100-0.25_7.dat-50, OR5x100-0.25_8.dat-0, OR10x500-0.25_5.dat-50, the point is one of Lefebvre and Schmidt's, from runs stopped at a time or solution limit, and lies below Lozano et al.'s lower bound. With one follower knapsack the median gap falls from 24.0% to 4.1% at budget 25% and from 34.0% to 7.1% at budget 50%; at budget 75% the published bounds were already tight. With several follower knapsacks the lower bounds are the literature's, so the gaps move only through better points: little at budget 25%, much more at 75%.

Table 6. The 16 knapsack interdiction instances without a published optimum. Class 6: subset sum ($p = w$); class 7: even weights, odd capacity. Index: the budget index of the instance name; β: the leader's budget; c: the follower's capacity. Every optimum equals c (class 6) or $c-1$ (class 7), the trivial upper bound, and BOBILib's value. Nodes and CPU seconds of the search on one thread; prover: nodes of the second prover.

Instance	Items	Class	Index	β	c	Optimum	Nodes	CPU (s)	Prover nodes
interKP-200-100-6-2	200	6	2	1,860	1,855	1,855	303	0.10	303
interKP-300-100-6-2	300	6	2	2,865	2,868	2,868	366	0.16	366
interKP-400-100-6-2	400	6	2	3,828	3,832	3,832	257	0.15	257
interKP-500-100-6-2	500	6	2	4,645	4,651	4,651	190	0.26	190
interKP-200-100-6-3	200	6	3	2,787	2,782	2,782	13,498	3.93	13,502
interKP-300-100-6-3	300	6	3	4,299	4,302	4,302	19,405	8.06	19,537
interKP-400-100-6-3	400	6	3	5,744	5,748	5,748	13,430	9.01	13,583
interKP-500-100-6-3	500	6	3	6,970	6,976	6,976	10,079	10.04	10,116
interKP-200-100-7-2	200	7	2	1,878	1,873	1,872	22	0.06	22
interKP-300-100-7-2	300	7	2	2,892	2,895	2,894	27	0.04	27
interKP-400-100-7-2	400	7	2	3,863	3,867	3,866	18	0.05	18
interKP-500-100-7-2	500	7	2	4,689	4,695	4,694	1	0.05	1
interKP-200-100-7-3	200	7	3	2,814	2,809	2,808	651	0.10	651
interKP-300-100-7-3	300	7	3	4,338	4,341	4,340	470	0.20	470
interKP-400-100-7-3	400	7	3	5,797	5,801	5,800	378	0.17	378
interKP-500-100-7-3	500	7	3	7,037	7,043	7,042	256	0.15	256

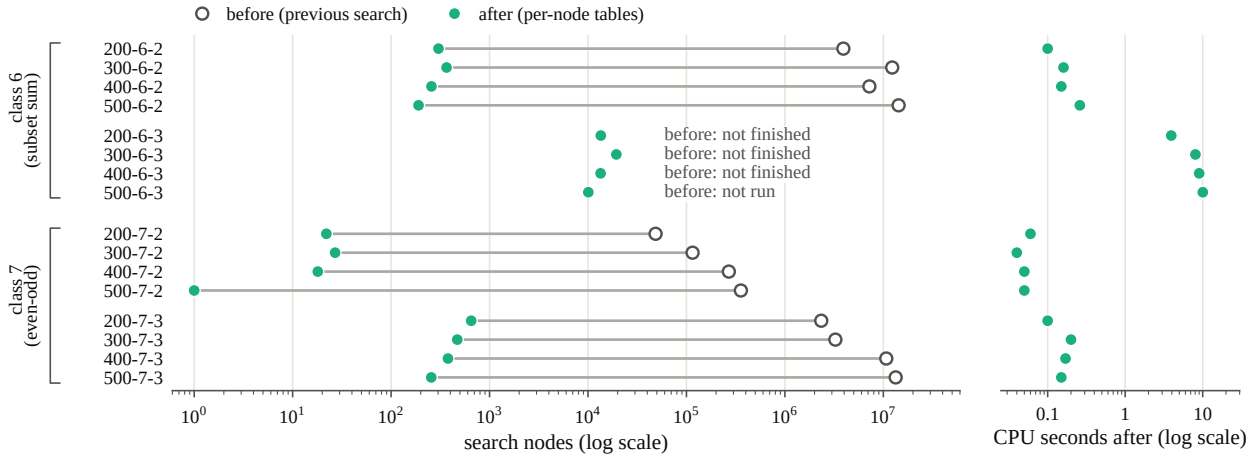


Figure 9. Search nodes for the 16 knapsack interdiction instances open in the literature, before and after the per-node tables (left, log scale), and CPU seconds after (right, log scale). Labels: items-class-index of `interKP-`*items*`-100-`*class*`-`*index*. All are proven optimal, the slowest (interKP-500-100-6-3, optimum 6,976) in 10,079 nodes and 10.0 seconds.

Of the 810 OR instances, 690 now have a verified point: all 270 with one follower knapsack, all 360 with several follower knapsacks at budget 50% or 75%, and 60 of the 180 at budget 25% (Figure 11). Compared with the best published point, BOBILib's included, 469 of these points are strictly better, 72 are equal and 61 are worse; for 88 instances the point is the first anywhere. For 52 more it is the first in BOBILib, although Lozano et al. had published one. With one follower knapsack the LP bracket and the merged-row online bound give a better lower bound than the published one for 191 instances.

One published optimum needs a note. For OR10x100-0.75_4.dat-0 Lozano, Bergman and Cire published a point of value 8,813 and, in their paper's table, a lower bound of 8,814 from runs they flag as numerically inconsistent; their repository also records a branch-and-cut bound of 8,813.[41] The point and the branch-and-cut bound prove the value optimal, so the instance is counted among the published optima. The verified solution file of this work at 8,813 (checked also with SCIP) confirms that the bound 8,814 is wrong.

Table 7. Median relative gap $(U-L)/U$ on the OR set over the instances with both a published lower and upper bound (paired; the 4 instances with conflicting published bounds left out), with the literature alone and with this work added, and the instances without any published point.

			Median gap (%)		No published point	
Follower knapsacks	Budget	Paired	Literature	Now	Instances	First point here
One (`-100`)	25%	90	24.0	4.1	0	0
	50%	90	34.0	7.1	0	0
	75%	90	0.14	0.14	0	0
Several (`-0`)	25%	54	33.3	31.8	31	0
	50%	59	36.6	29.1	31	31
	75%	62	5.3	0.05	27	27
Several (`-50`)	25%	68	30.6	28.3	20	0
	50%	70	41.3	34.8	20	20
	75%	80	11.8	1.4	10	10

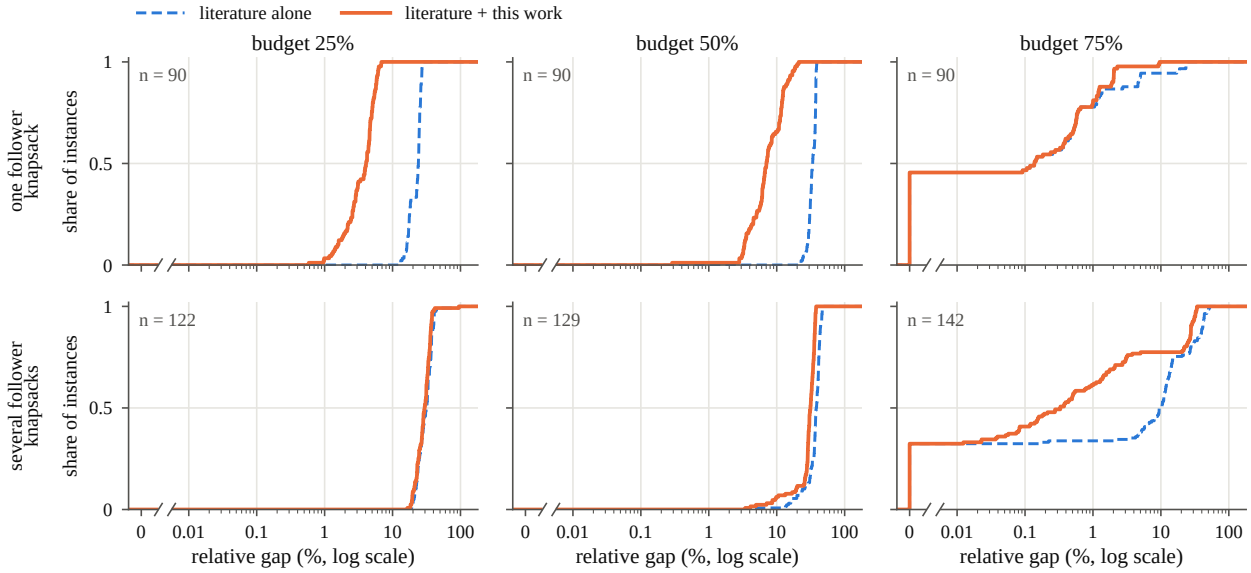


Figure 10. Relative optimality gap $(U-L)/U$ on the OR set over the instances with both published bounds (the 4 with conflicting published bounds left out), literature alone against literature plus this work, by follower type and budget (cumulative share; n: instances in the panel). Gaps of 0, closed instances, are drawn at the separate "0" tick; no gap is negative.

4.5 Clique Interdiction

Of the 60 clique instances, 57 are optimal (Figure 12): 27 at density 0.7 with value 3 and all 30 at density 0.9 with value 4, where BOBILib lists 5 to 13. BOBILib's points are valid feasible points; their leader choices are just far from optimal. The 3 open instances, normalClique60-70-2, normalClique60-70-3, normalClique60-70-5, have value 3 or 4. Value 3 needs a set of at most $k = 309$ edges that meets every K_4. The best 3-partitions found have 311, 310 and 310 inside edges, and searches that are not restricted to 3-partite remainders stop at the same values. A proof of value 4 cannot use Brouwer's theorem: $m - k = 930$ is far below its threshold 1,182, and there are K_4-free graphs with that many edges that are not 3-partite (Supp. Sec. D.6). LP bounds on the smallest such edge set stop near 296.

4.6 Network and General Instances

The network instances tree-50_1-5-5_006 and tree-50_1-5-5_010 are optimal at 26, BOBILib's value; 4 and 28 of the 2,138,410 attacks reach it. Of BOBILib's 70 optimal network values, 54 are derived again: the 34 trees by the same code, the 12 random graphs with 25 nodes by HiGHS at every attack, and 8 random graphs with 50 or 80 nodes by the brute force of the separate review.

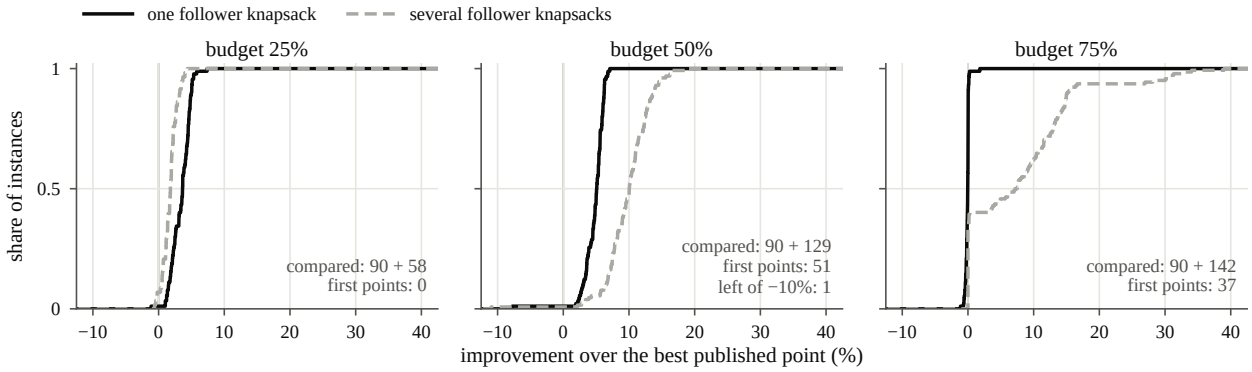


Figure 11. Improvement of the verified OR points over the best published point (literature or BOBILib), by budget and follower type (cumulative share). In each panel, "compared" gives the points compared with one and with several follower knapsacks, "first points" the instances with no published point (not shown), and "left of −10%" the points off the scale on the left; the three points of instances with conflicting published bounds are left out. Of the 690 points, 469 are strictly better.

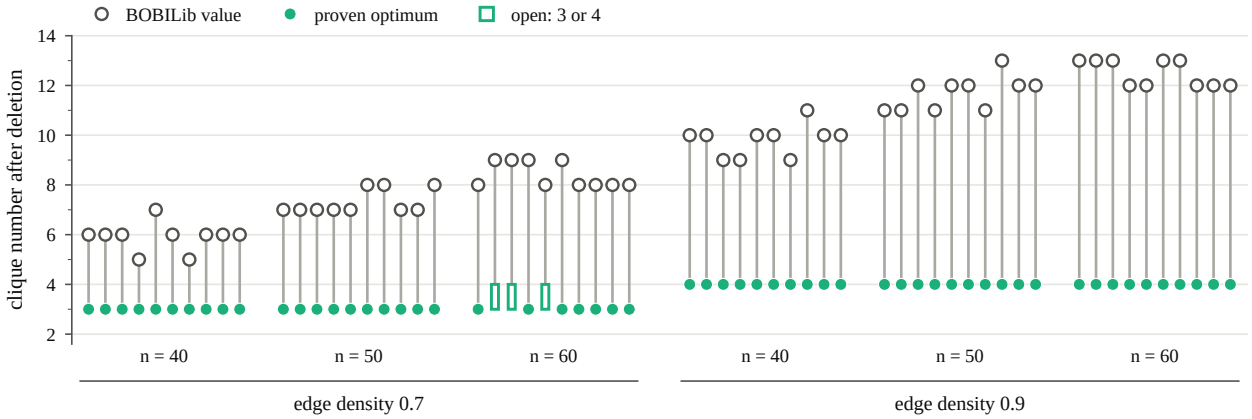


Figure 12. The 60 clique interdiction instances: BOBILib's value (5 to 13, open circle) against the proven optimum (3 at density 0.7, 4 at density 0.9). The 3 open instances lie between 3 and 4.

Figure 13 shows the outcome for the 266 unsolved general instances. Of the instances that BOBILib lists as open, 40 have no bilevel feasible point: 38 certificates pass the exact rational check, and for the other 2 (supportcase42_0_100 and supportcase42_50_50) small rounding residuals remain, so they rest on floating point. Without coupling rows, `bobi general` was run on 98 open instances. 17 of them have no bilevel feasible point, and of the other 81 it gives verified points for 64. For 9 of these a point had been published elsewhere, better than ours for 6 (Supp. Table S12); for the others it is the first point known. With coupling rows, `bobi coupled` was run on 52 of 54 candidates and found 29 verified points: 21 at the HPR's own leader values, 5 from the penalized HPR and 3 from best-response dynamics. In 9 of these files the follower's value lies about 10^{-7} relative above its optimum, within BOBILib's test (3), and the leader uses that slack. The files for pg_50_50 and neos-2978193-inde_50_50 are feasible only within that tolerance, although both instances do have exactly bilevel feasible points (Supp. Sec. E.4).

Table 8 lists the 5 instances that are optimal at the MIPLIB bound. Of these, pigeon-10_0_100, pigeon-10_50_50 and timtab1_0_100 were open, with no point known. For noswot_0_100 and noswot_50_50 BOBILib's own points already had the optimal value; the proof is new. The generalized interdiction instances that remain unchanged (8 of them) have many linking variables: enumerating their binary linking choices, which reproduces a published optimum on a smaller instance of the set, is too slow for them.

5. DISCUSSION

The new results come from structure, certificates and published bounds that a general-purpose MIBLP solver does not use. A general branch and bound over the 1,593 edge variables of a 60-vertex clique instance at density 0.9 is unlikely to show

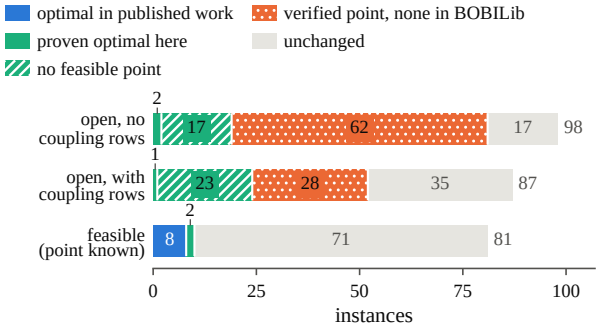


Figure 13. Outcomes for the 266 general instances BOBILib lists as feasible or open, by BOBILib status and, for the open ones, by whether leader rows contain follower variables (coupling rows).

Table 8. Instances whose HPR is exactly the MIPLIB model they were built from, with a verified point at its published optimum. BOBILib: status before this work; point from: BOBILib's solution file or the method of this work. MIPLIB's optimum of noswot, −41.00000885, rounds up because its objective takes integer values only.

Instance	BOBILib	Optimum	Point from
noswot_0_100	feasible	−41	BOBILib
noswot_50_50	feasible	−41	BOBILib
pigeon-10_0_100	open	−9,000	`bobi general`
timtab1_0_100	open	764,771.99999978	`bobi general`
pigeon-10_50_50	open	−9,000	`bobi coupled`

quickly that every deletion of 398 edges leaves a K_4, while Brouwer's theorem and 170 small cases do; and an instance without a bilevel feasible point needs one direction as a certificate, while a solver that searches for a point can only report a time limit.

5.1 Recommendations for Benchmark Libraries

Benchmark statuses are most useful when they are current and when each comes with its evidence. From this work I suggest:

- record the provenance of every status (a solver run with its limits, a published proof, or a certificate), and import published per-instance results, so that an instance solved elsewhere is not listed as unsolved;
- screen new instances for followers that are infeasible or unbounded for every leader choice, and accept certificates of bilevel infeasibility, such as the recession directions here, as evidence for an infeasible status;
- state the tolerances behind every status, and prefer exactly optimal follower responses. In BOBILib's solution file for the generalized interdiction instance general20-30-20-20-20-2 the follower's value is 106,899,984.14, while the follower's optimum at the same leader choice is 106,899,953: 31 units, or 2.9×10^{-7} relative, above it. Test (3) accepts this because the follower's values are large, and the rows, violated by up to 3.3×10^{-4}, pass the 10^{-4} tolerance of the MIPLIB checker inside BOBILib's checker (Supp. Sec. A.2). `bobi verify` rejects the file: it does not meet the rows within 10^{-6}, and its stated objective 100,056 differs from the 100,056.207 of its point (Supp. Sec. E.5). This work is not free of the issue either: 9 of its coupled files use the follower slack of Table 2, and with an exactly optimal follower their leader values are slightly worse (listed in the data package);
- distribute solution files and certificates in standard formats, with a status file in the MIPLIB `.solu` format, as the data package of this paper does.

5.2 Limitations and Threats to Validity

Running times. All times come from a shared four-core machine that was often oversubscribed; they show orders of magnitude, not careful benchmarks.

Floating point and tolerances. The points are feasible within 10^{-6} and follower-optimal within BOBILib's relative tolerance, not exactly, and 9 coupled points use that slack (Section 4.6). Of the 40 recession certificates, 2 are checked only in floating point; the status file marks them. The MIPLIB optima behind the 5 bound-based optima are published floating-point values; timtab1_0_100, for example, is optimal at 764,771.99999978, the value MIPLIB prints as 764,772. The LP bracket and the merged-row bound are evaluated in double precision, and the window rule of the subset-sum search

uses a floating-point fractional bound; the 16 subset-sum proofs are confirmed by the integer-only provers, which do not use that rule.

Trusted solvers. Follower optimality for followers with several rows and for general instances, the 170 part-size verdicts and the dual bounds of the OR bracket come from MILP and constraint programming solvers. The part-size verdicts were reproduced by several methods, two of them solver-free, but a bug that all methods share would go unnoticed. Certificates in a checkable proof format, such as VIPR for MILP verdicts[69] from an exact rational solver[70] or pseudo-Boolean proof logs checked by VeriPB,[71] would remove this trust; they were not produced here.

Scope of the Lean proofs. Lean covers the main pruning lemmas of the KIP search and the Mantel and partition lemmas for clique interdiction. It does not cover the reduction to the count formulation, the cheaper rules of Supp. Thm B.23, Brouwer's theorem, the LP bracket, the network DP, the recession directions, or the search code itself (Table 4).

Results taken from the literature. The 528 published optima, the work of their authors, and the published lower bounds of the OR instances with several follower knapsacks are used as reported; for 82 of the optima not even a point is verified here. "First point anywhere" means that none of the published sources compiled here (Supp. Sec. H) has a point; for 10 of the general instances given a point by `bobi general` or `bobi coupled` a published point exists (for 4 of them only in Lefebvre and Schmidt's run files), and these are not counted as first points. Other sources may hold points that this reconciliation missed.

Common-mode errors. The solvers, the verifier and the separate review sessions were all written with the same AI coding agent directed by the same author. Separate sessions with their own code and instance readers reduce, but do not remove, the risk of a shared misreading of the file format or a shared conceptual slip. The Lean proofs, the exact rational checks and the brute-force comparisons are the checks least exposed to it.

Instance reading. Every argument depends on reading the instance files correctly. Each family's model was checked row by row against its expected pattern, and the key results were derived again with separately written readers.

5.3 What Stays Hard

For the 3 open clique instances, value 3 needs a set of at most k edges that meets every K_4, which no search has found, and value 4 needs a lower bound on the size of such a set well above the LP bounds. For random graphs the largest K_4-free subgraphs are 3-partite,[72,73] which would point to value 4, but these results are asymptotic and give no bound for a given graph with 60 vertices. The OR instances with several follower knapsacks at budget 25% still have no certified point in 120 cases: 51 have no point anywhere, and 69 have only a published, unverified point. Their median gaps stay near 31.8% and 28.3%; they need a lower bound that uses the structure of several knapsacks, which the one-row bracket does not. Generalized interdiction with many linking variables needs a pruned enumeration of the linking choices, and 123 general instances (122 of them MIPLIB-derived) remain without a point or without a proof. The results of this work will be submitted to BOBILib's maintainers, who update the statuses from submitted solutions.

6. CONCLUSIONS

I asked which of BOBILib's 1,594 feasible or open instances can be settled. The key findings are:

- Published work had already solved 528 of them; reconciling BOBILib's statuses, which predate some of these results, with the literature settles more instances than any single method.
- Structure-specific methods settle 120 more: a robust subset-sum search proves all 16 remaining knapsack interdiction instances optimal; extremal graph theory, with partition certificates and 170 cases decided by exact solvers, settles 57 of 60 clique interdiction instances; recession directions show that 40 instances have no bilevel feasible point (38 checked exactly); published MIPLIB optima prove 5 more optimal; and exhaustive enumeration with an exact tree dynamic program settles the 2 network interdiction instances.
- In all, 648 instances (41%) are settled. 230 instances get a verified point where BOBILib lists none, 168 of them the first point anywhere, and 469 OR instances get a point better than any published one.
- The claims can be checked: solution files in BOBILib's format pass a separate verifier modeled on BOBILib's checker, the certificates are published, and the main pruning lemmas of the new search are proved in Lean.

DATA AVAILABILITY

The data package is the folder `results/` of the repository REPOSITORY-URL (archived with DOI DATA-DOI). It contains `instances.csv`, one row per BOBILib instance with BOBILib's status and value, the new status, the best verified value and proven bound, the source, the method and the certificate; `bobilib_update.solu`, the status changes in MIPLIB's `.solu` format (608 `=opt=`, 40 `=inf=`, 695 `=best=` and 194 `=bestdual=` lines; its header marks the 2 `=inf=` lines that rest on floating-point certificates); the solution files in BOBILib's format; the certificates (recession directions, clique partitions with the solver logs of the part-size proofs, subset-sum proof records, enumeration summaries, OR bounds); and `SHA256SUMS`, the checksums of every file. The code is in the same repository: the Rust crate `bobi` (the solvers and the verifier), the scripts of the separate checks, and the Lean project with the proofs. The supplementary document (`paper/appendix/` in the repository) holds the proofs, the verification protocol, the map of the Lean theorems and the per-instance tables.

ACKNOWLEDGEMENTS

The software development, the computational experiments, the separate review sessions and the drafting of this paper were carried out with substantial assistance from an artificial intelligence (AI) coding agent (Claude Code, Anthropic) directed by the author. The reviews described in Section 3.7 were separate sessions of the same agent that wrote their own code. All results were checked by the verification described in Sections 3.1 and 3.7. I thank the maintainers of BOBILib, Johannes Thürauf, Thomas Kleinert, Ivana Ljubić, Ted Ralphs and Martin Schmidt, for the library, its solution files and its checker, and the authors who published their per-instance data and code, in particular Noah Weninger and Ricardo Fukasawa; Matteo Fischetti, Ivana Ljubić, Michele Monaci and Markus Sinnl; Leonardo Lozano, David Bergman and Andre A. Cire; and Henri Lefebvre and Martin Schmidt.

REFERENCES

[1] von Stackelberg, H., *Market Structure and Equilibrium*, Springer, Berlin (2011). English translation by D. Bazin et al. of *Marktform und Gleichgewicht*, Julius Springer, Vienna, 1934. https://doi.org/10.1007/978-3-642-12586-7.

[2] Brown, G., Carlyle, M., Salmerón, J. and Wood, K., "Defending critical infrastructure," *Interfaces* **36**(6), 530–544 (2006). https://doi.org/10.1287/inte.1060.0252.

[3] Smith, J. C. and Song, Y., "A survey of network interdiction models and algorithms," *European Journal of Operational Research* **283**(3), 797–811 (2020). https://doi.org/10.1016/j.ejor.2019.06.024.

[4] Jeroslow, R. G., "The polynomial hierarchy and a simple model for competitive analysis," *Mathematical Programming* **32**(2), 146–164 (1985). https://doi.org/10.1007/BF01586088.

[5] Caprara, A., Carvalho, M., Lodi, A. and Woeginger, G. J., "A study on the computational complexity of the bilevel knapsack problem," *SIAM Journal on Optimization* **24**(2), 823–838 (2014). https://doi.org/10.1137/130906593.

[6] Dempe, S., *Foundations of Bilevel Programming*, Nonconvex Optimization and Its Applications, Kluwer Academic Publishers (2002). https://doi.org/10.1007/b101970.

[7] Kleinert, T., Labbé, M., Ljubić, I. and Schmidt, M., "A survey on mixed-integer programming techniques in bilevel optimization," *EURO Journal on Computational Optimization* **9**, 100007 (2021). https://doi.org/10.1016/j.ejco.2021.100007.

[8] Bixby, R. E., Ceria, S., McZeal, C. M. and Savelsbergh, M. W. P., "An updated mixed integer programming library: MIPLIB 3.0," *Optima* **58**, 12–15 (1998). https://miplib2010.zib.de/miplib3/miplib.html.

[9] Koch, T., Achterberg, T., Andersen, E., Bastert, O., Berthold, T., Bixby, R. E., Danna, E., Gamrath, G., Gleixner, A. M., Heinz, S., Lodi, A., Mittelmann, H., Ralphs, T., Salvagnin, D., Steffy, D. E. and Wolter, K., "MIPLIB 2010," *Mathematical Programming Computation* **3**(2), 103–163 (2011). https://doi.org/10.1007/s12532-011-0025-9.

[10] Gleixner, A., Hendel, G., Gamrath, G., Achterberg, T., Bastubbe, M., Berthold, T., Christophel, P., Jarck, K., Koch, T., Linderoth, J., Lübbecke, M., Mittelmann, H. D., Ozyurt, D., Ralphs, T. K., Salvagnin, D. and Shinano, Y., "MIPLIB 2017: data-driven compilation of the 6th mixed-integer programming library," *Mathematical Programming Computation* **13**(3), 443–490 (2021). https://doi.org/10.1007/s12532-020-00194-3.

[11] Thürauf, J., Kleinert, T., Ljubić, I., Ralphs, T. K. and Schmidt, M., "BOBILib: Bilevel Optimization (Benchmark) Instance Library," *Mathematical Programming Computation* **18**(3), 819–876 (2026). https://doi.org/10.1007/s12532-025-00294-y.

[12] Thürauf, J., Kleinert, T., Ljubić, I., Ralphs, T. K. and Schmidt, M., "BOBILib: Bilevel Optimization (Benchmark) Instance Library," Website with instances, solution files, statuses and feasibility checker (2026). Accessed 18 October 2026. https://bobilib.org.

[13] Tahernejad, S., Ralphs, T. K. and DeNegre, S. T., "A branch-and-cut algorithm for mixed integer bilevel linear optimization problems and its implementation," *Mathematical Programming Computation* **12**(4), 529–568 (2020). https://doi.org/10.1007/s12532-020-00183-6.

[14] DeNegre, S. T., Ralphs, T. K., Tahernejad, S. and Xie, Y., "MibS," Open-source solver for mixed integer bilevel linear optimization, COIN-OR; BOBILib's statuses use version 1.2.2 (2024). https://github.com/coin-or/MibS.

[15] Fischetti, M., Ljubić, I., Monaci, M. and Sinnl, M., "A new general-purpose algorithm for mixed-integer bilevel linear programs," *Operations Research* **65**(6), 1615–1637 (2017). https://doi.org/10.1287/opre.2017.1650.

[16] Fischetti, M., Ljubić, I., Monaci, M. and Sinnl, M., "Bilevel integer programming and interdiction problems," Web page with the intersection-cut solver for mixed-integer bilevel linear problems (MIX++ setting) and instance sets (2024). Accessed 18 October 2026. https://msinnl.github.io/pages/bilevel.html.

[17] Weninger, N. and Fukasawa, R., "A fast combinatorial algorithm for the bilevel knapsack problem with interdiction constraints," *Mathematical Programming* **210**(1–2), 847–879 (2025). https://doi.org/10.1007/s10107-024-02133-9.

[18] Fischetti, M., Ljubić, I., Monaci, M. and Sinnl, M., "Interdiction games and monotonicity, with application to knapsack problems," *INFORMS Journal on Computing* **31**(2), 390–410 (2019). https://doi.org/10.1287/ijoc.2018.0831.

[19] Lozano, L., Bergman, D. and Cire, A. A., "Network relaxations for combinatorial bilevel optimization under linear interactions," *Operations Research* (2026). Articles in Advance, published online 26 August 2026. https://doi.org/10.1287/opre.2025.2457.

[20] Caprara, A., Carvalho, M., Lodi, A. and Woeginger, G. J., "Bilevel knapsack with interdiction constraints," *INFORMS Journal on Computing* **28**(2), 319–333 (2016). https://doi.org/10.1287/ijoc.2015.0676.

[21] Wood, R. K., "Deterministic network interdiction," *Mathematical and Computer Modelling* **17**(2), 1–18 (1993). https://doi.org/10.1016/0895-7177(93)90236-R.

[22] Israeli, E. and Wood, R. K., "Shortest-path network interdiction," *Networks* **40**(2), 97–111 (2002). https://doi.org/10.1002/net.10039.

[23] Kellerer, H., Pferschy, U. and Pisinger, D., *Knapsack Problems*, Springer, Berlin (2004). https://doi.org/10.1007/978-3-540-24777-7.

[24] Meyer, R. R., "On the existence of optimal solutions to integer and mixed-integer programming problems," *Mathematical Programming* **7**(1), 223–235 (1974). https://doi.org/10.1007/BF01585518.

[25] Moore, J. T. and Bard, J. F., "The mixed integer linear bilevel programming problem," *Operations Research* **38**(5), 911–921 (1990). https://doi.org/10.1287/opre.38.5.911.

[26] DeNegre, S., *Interdiction and Discrete Bilevel Linear Programming*. PhD thesis, Lehigh University, Bethlehem, PA (2011). https://coral.ise.lehigh.edu/~ted/files/papers/ScottDeNegreDissertation11.pdf.

[27] Fischetti, M., Monaci, M. and Sinnl, M., "A dynamic reformulation heuristic for Generalized Interdiction Problems," *European Journal of Operational Research* **267**(1), 40–51 (2018). https://doi.org/10.1016/j.ejor.2017.11.043.

[28] Martello, S., Pisinger, D. and Toth, P., "Dynamic programming and strong bounds for the 0-1 knapsack problem," *Management Science* **45**(3), 414–424 (1999). https://doi.org/10.1287/mnsc.45.3.414.

[29] Khuri, S., Bäck, T. and Heitkötter, J., "The zero/one multiple knapsack problem and genetic algorithms," in *Proceedings of the 1994 ACM Symposium on Applied Computing (SAC '94)*, 188–193, ACM, New York, NY (1994). https://doi.org/10.1145/326619.326694.

[30] Chu, P. C. and Beasley, J. E., "A genetic algorithm for the multidimensional knapsack problem," *Journal of Heuristics* **4**(1), 63–86 (1998). https://doi.org/10.1023/A:1009642405419.

[31] Beasley, J. E., "OR-Library: distributing test problems by electronic mail," *Journal of the Operational Research Society* **41**(11), 1069–1072 (1990). https://doi.org/10.1057/jors.1990.166.

[32] Tang, Y., Richard, J.-P. P. and Smith, J. C., "A class of algorithms for mixed-integer bilevel min-max optimization," *Journal of Global Optimization* **66**(2), 225–262 (2016). https://doi.org/10.1007/s10898-015-0274-7.

[33] Baggio, A., Carvalho, M., Lodi, A. and Tramontani, A., "Multilevel approaches for the critical node problem," *Operations Research* **69**(2), 486–508 (2021). https://doi.org/10.1287/opre.2020.2014.

[34] Fischetti, M., Ljubić, I., Monaci, M. and Sinnl, M., "Intersection cuts for bilevel optimization," in *Integer Programming and Combinatorial Optimization (IPCO 2016)*, Louveaux, Q. and Skutella, M., eds., *Lecture Notes in Computer Science* **9682**, 77–88, Springer, Cham (2016). https://doi.org/10.1007/978-3-319-33461-5_7.

[35] Fischetti, M., Ljubić, I., Monaci, M. and Sinnl, M., "On the use of intersection cuts for bilevel optimization," *Mathematical Programming* **172**(1–2), 77–103 (2018). https://doi.org/10.1007/s10107-017-1189-5.

[36] Kleinert, T. and Schmidt, M., "Computing feasible points of bilevel problems with a penalty alternating direction method," *INFORMS Journal on Computing* **33**(1), 198–215 (2021). https://doi.org/10.1287/ijoc.2019.0945.

[37] Xu, P. and Wang, L., "An exact algorithm for the bilevel mixed integer linear programming problem under three simplifying assumptions," *Computers & Operations Research* **41**, 309–318 (2014). https://doi.org/10.1016/j.cor.2013.07.016.

[38] Ozaltin, O. Y. and Zhang, J., "A branch-and-cut algorithm for discrete bilevel linear programs," tech. rep., Optimization Online (2017). https://optimization-online.org/?p=14558.

[39] Della Croce, F. and Scatamacchia, R., "An exact approach for the bilevel knapsack problem with interdiction constraints and extensions," *Mathematical Programming* **183**(1–2), 249–281 (2020). https://doi.org/10.1007/s10107-020-01482-5.

[40] Weninger, N. and Fukasawa, R., "A fast combinatorial algorithm for the bilevel knapsack problem with interdiction constraints," in *Integer Programming and Combinatorial Optimization (IPCO 2023)*, Del Pia, A. and Kaibel, V., eds., *Lecture Notes in Computer Science* **13904**, 438–452, Springer, Cham (2023). https://doi.org/10.1007/978-3-031-32726-1_31.

[41] Lozano, L., Bergman, D. and Cire, A. A., "DiscreteBilevelOptimization: code, computational results and BOBILib solution files for 'network relaxations for combinatorial bilevel optimization under linear interactions'," Source code and data (2026). https://github.com/andrecire/DiscreteBilevelOptimization.

[42] Weninger, N. and Fukasawa, R., "bkpsolver: code accompanying 'A fast combinatorial algorithm for the bilevel knapsack problem with interdiction constraints'," Source code, instances and answer files (MIT license) (2026). https://github.com/nwoeanhinnogaehr/bkpsolver.

[43] Lefebvre, H. and Schmidt, M., "A Dantzig–Wolfe single-level reformulation for mixed-integer linear bilevel optimization: exact and heuristic approaches," preprint, Optimization Online (June 2025). Per-instance results on the first author's web page, henrilefebvre.com. https://optimization-online.org/2025/06/a-dantzig-wolfe-single-level-reformulation-for-mixed-integer-linear-bilevel-optimization-exact-and-heuristic-approaches/.

[44] Furini, F., Ljubić, I., Martin, S. and San Segundo, P., "The maximum clique interdiction problem," *European Journal of Operational Research* **277**(1), 112–127 (2019). https://doi.org/10.1016/j.ejor.2019.02.028.

[45] Furini, F., Ljubić, I., San Segundo, P. and Zhao, Y., "A branch-and-cut algorithm for the Edge Interdiction Clique Problem," *European Journal of Operational Research* **294**(1), 54–69 (2021). https://doi.org/10.1016/j.ejor.2021.01.030.

[46] Mahdavi Pajouh, F., "Minimum cost edge blocker clique problem," *Annals of Operations Research* **294**(1–2), 345–376 (2020). https://doi.org/10.1007/s10479-019-03315-x.

[47] DeNegre, S. T. and Ralphs, T. K., "A branch-and-cut algorithm for integer bilevel linear programs," in *Operations Research and Cyber-Infrastructure*, Chinneck, J. W., Kristjansson, B. and Saltzman, M. J., eds., *Operations Research/Computer Science Interfaces* **47**, 65–78, Springer, Boston, MA (2009). https://doi.org/10.1007/978-0-387-88843-9_4.

[48] Mitsos, A., "Global solution of nonlinear mixed-integer bilevel programs," *Journal of Global Optimization* **47**(4), 557–582 (2010). https://doi.org/10.1007/s10898-009-9479-y.

[49] Lozano, L. and Smith, J. C., "A value-function-based exact approach for the bilevel mixed-integer programming problem," *Operations Research* **65**(3), 768–786 (2017). https://doi.org/10.1287/opre.2017.1589.

[50] Vásquez, S., Lozano, L. and van Hoeve, W.-J., "A single-level reformulation of binary bilevel programs using decision diagrams," *Mathematical Programming* (2025). Published online 16 December 2025. https://doi.org/10.1007/s10107-025-02294-1.

[51] Dantzig, G. B., "Discrete-variable extremum problems," *Operations Research* **5**(2), 266–288 (1957). https://doi.org/10.1287/opre.5.2.266.

[52] Martello, S. and Toth, P., *Knapsack Problems: Algorithms and Computer Implementations*, Wiley-Interscience Series in Discrete Mathematics and Optimization, John Wiley & Sons, Chichester (1990).

[53] Huangfu, Q. and Hall, J. A. J., "Parallelizing the dual revised simplex method," *Mathematical Programming Computation* **10**(1), 119–142 (2018). https://doi.org/10.1007/s12532-017-0130-5.

[54] Mantel, W., "Problem 28 (solution by H. Gouwentak, W. Mantel, J. Teixeira de Mattes, F. Schuh and W. A. Wythoff)," *Wiskundige Opgaven* **10**, 60–61 (1907).

[55] Brouwer, A. E., "Some lotto numbers from an extension of Turán's theorem," Afdeling Zuivere Wiskunde report ZW 152/81, Mathematisch Centrum, Amsterdam (Mar. 1981). https://ir.cwi.nl/pub/6791.

[56] Tyomkyn, M. and Uzzell, A. J., "Strong Turán stability," *The Electronic Journal of Combinatorics* **22**(3), P3.9 (2015). https://doi.org/10.37236/4717.

[57] Turán, P., "Egy gráfelméleti szélsőértékfeladatról [On an extremal problem in graph theory]," *Matematikai és Fizikai Lapok* **48**, 436–452 (1941). In Hungarian.

[58] Hanson, D. and Toft, B., "k-saturated graphs of chromatic number at least k," *Ars Combinatoria* **31**, 159–164 (1991).

[59] Kang, M. and Pikhurko, O., "Maximum K_{r+1}-free graphs which are not r-partite," *Matematychni Studii* **24**(1), 12–20 (2005).

[60] Amin, K., Faudree, J., Gould, R. J. and Sidorowicz, E., "On the non-$(p-1)$-partite K_p-free graphs," *Discussiones Mathematicae Graph Theory* **33**(1), 9–23 (2013). https://doi.org/10.7151/dmgt.1654.

[61] Perron, L. and Didier, F., "CP-SAT, version 9.15," Google OR-Tools (2026). https://developers.google.com/optimization/cp/cp_solver/.

[62] Hojny, C., Besançon, M., Bestuzheva, K., Borst, S., Chmiela, A., Dionísio, J., Eifler, L., Ghannam, M., Gleixner, A., Göß, A., Hoen, A., van der Hulst, R., Kamp, D., Koch, T., Kofler, K., Lentz, J., Maher, S. J., Mexi, G., Mühmer, E., Pfetsch, M. E., Pokutta, S., Serrano, F., Shinano, Y., Turner, M., Vigerske, S., Walter, M., Weninger, D. and Xu, L., "The SCIP Optimization Suite 10.0," technical report, Optimization Online (Nov. 2025). https://optimization-online.org/2025/11/the-scip-optimization-suite-10-0/.

[63] Maher, S., Miltenberger, M., Pedroso, J. P., Rehfeldt, D., Schwarz, R. and Serrano, F., "PySCIPOpt: mathematical programming in Python with the SCIP Optimization Suite," in *Mathematical Software – ICMS 2016*, *Lecture Notes in Computer Science* **9725**, 301–307, Springer, Cham (2016). https://doi.org/10.1007/978-3-319-42432-3_37.

[64] Bron, C. and Kerbosch, J., "Algorithm 457: finding all cliques of an undirected graph," *Communications of the ACM* **16**(9), 575–577 (1973). https://doi.org/10.1145/362342.362367.

[65] Tomita, E., Tanaka, A. and Takahashi, H., "The worst-case time complexity for generating all maximal cliques and computational experiments," *Theoretical Computer Science* **363**(1), 28–42 (2006). https://doi.org/10.1016/j.tcs.2006.06.015.

[66] Carraghan, R. and Pardalos, P. M., "An exact algorithm for the maximum clique problem," *Operations Research Letters* **9**(6), 375–382 (1990). https://doi.org/10.1016/0167-6377(90)90057-C.

[67] de Moura, L. and Ullrich, S., "The Lean 4 theorem prover and programming language," in *Automated Deduction – CADE 28*, Platzer, A. and Sutcliffe, G., eds., *Lecture Notes in Computer Science* **12699**, 625–635, Springer, Cham (2021). https://doi.org/10.1007/978-3-030-79876-5_37.

[68] The mathlib Community, "The Lean mathematical library," in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, 367–381, ACM, New York, NY (2020). https://doi.org/10.1145/3372885.3373824.

[69] Cheung, K. K. H., Gleixner, A. and Steffy, D. E., "Verifying integer programming results," in *Integer Programming and Combinatorial Optimization (IPCO 2017)*, Eisenbrand, F. and Koenemann, J., eds., *Lecture Notes in Computer Science* **10328**, 148–160, Springer, Cham (2017). https://doi.org/10.1007/978-3-319-59250-3_13.

[70] Eifler, L. and Gleixner, A., "A computational status update for exact rational mixed integer programming," *Mathematical Programming* **197**(2), 793–812 (2023). https://doi.org/10.1007/s10107-021-01749-5.

[71] Bogaerts, B., Gocht, S., McCreesh, C. and Nordström, J., "Certified dominance and symmetry breaking for combinatorial optimisation," *Journal of Artificial Intelligence Research* **77**, 1539–1589 (2023). https://doi.org/10.1613/jair.1.14296.

[72] Babai, L., Simonovits, M. and Spencer, J., "Extremal subgraphs of random graphs," *Journal of Graph Theory* **14**(5), 599–622 (1990). https://doi.org/10.1002/jgt.3190140511.

[73] Brightwell, G., Panagiotou, K. and Steger, A., "Extremal subgraphs of random graphs," *Random Structures & Algorithms* **41**(2), 147–178 (2012). https://doi.org/10.1002/rsa.20413.